

Rapport de stage

Amélioration de la résolution de puzzles logiques par un pipeline hybride LLM + solveur formel

Alaaeddine Ben Salah
aladdin.bensalah2004 [at] gmail [dot] com

1 Introduction

Ce rapport présente les travaux réalisés jusqu'à présent dans le cadre de mon stage, dont l'objectif est d'améliorer la capacité des modèles de langage (LLM) à résoudre des puzzles logiques du benchmark Zebra en les associant à des solveurs formels via un langage intermédiaire spécifique (DSL).

2 Évaluation des performances du modèle de langage seul

La première étape consiste à évaluer la performance brute du LLM : Llama v3.3 70B sans assistance extérieure. Les résultats obtenus sur le benchmark Zebra sont présentés dans le tableau 1.

TABLE 1 – Résultats du modèle seul selon la taille des puzzles

Catégorie	Succès	Échecs	Taux de réussite (%)	Temps moyenne (m)
Small	173	147	54.6	0.22
Medium	18	262	6.4	0.37
Large	1	199	0.5	0.48
X-Large	0	200	0.0	0.67
Total	192	808	19.2	6.74 (h)

Le taux global de réussite du modèle seul est de 19,2%, ce qui met en évidence ses limites sur ce benchmark complexe.

3 Architecture du pipeline et méthode

Le pipeline développé s'appuie sur une interaction entre le puzzle logique initial, une génération de programme en langage spécifique (DSL), un solveur de contraintes (CBMC, Prolog), et la production finale de la solution. La Figure 1 illustre ce flux.

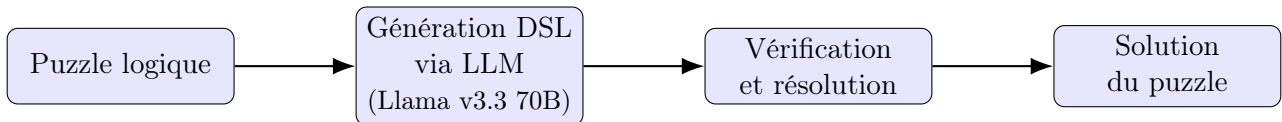


FIGURE 1 – Schéma du pipeline de résolution de puzzles logiques

3.1 Méthodologie de génération DSL avec LLM

La méthode repose sur une série de prompts successifs, chacun destiné à générer une étape précise du programme DSL compatible avec le solveur CBMC. Trois étapes principales sont distinguées :

1. **Génération de la structure de données** : Le premier prompt demande au modèle LLM (Llama v3.3 70B) de générer une structure de classes représentant les entités du puzzle logique (personne, maison, etc.). Cette structure repose sur trois constructions principales du DSL :
 - **Unique** : pour spécifier que les éléments d’une liste sont distincts ;
 - **Domain** : pour restreindre les valeurs possibles d’un champ ;
 - **List** : pour définir une collection d’objets de taille fixe.

Exemple :

```
class house:
    name: Unique[Domain[str, "peter", "eric", "arnold"]]
    hobby: Unique[Domain[str, "photography", "cooking", "gardening"]]

class solution:
    houses: list[house, 3]
```

2. **Génération des contraintes** : Une fois la structure de données définie, un second prompt est utilisé pour produire le code des contraintes logiques à partir des énoncés du puzzle. Ce code utilise les constructions du DSL inspirées du paradigme de vérification formelle :
 - **nondet(...)** : pour sélectionner une instance arbitraire (non déterministe) de la solution ;
 - **assume(...)** : pour imposer une propriété sur cette instance ;
 - **assert(...)** : pour exprimer la contrainte logique à vérifier.

Exemple :

```
# Clue 1: Eric is the person who is a teacher.
eric = nondet(solution.houses)
assume(eric.name == "Eric")
assert eric.occupation == "teacher"

# Clue 2: Alice is not in the second house.
alice = nondet(solution.houses)
assume(alice.name == "Alice")
assert solution.houses.index(alice) != 1
```

Cette structure constitue une abstraction intermédiaire, conçue pour être facilement manipulable par le LLM. Elle est ensuite automatiquement transformée en code C à l’aide de la bibliothèque `libcst`, afin d’être compatible avec le solveur formel CBMC.

3. **Formatage de la solution du solveur** : Une fois la solution calculée par CBMC, un dernier prompt est utilisé pour demander au LLM de transformer cette solution brute en un format structuré. Le LLM reçoit en entrée la solution brute et le format cible, et doit produire une réponse structurée conforme au format demandé.

Les résultats détaillés sont donnés au tableau 2.

TABLE 2 – Résultats du pipeline hybride LLM + CBMC selon la taille des puzzles

Catégorie	Succès	Échecs	Taux de réussite (%)	Temps moyenne (m)
Small	309	11	96.6	1.07
Medium	258	22	92.2	1.95
Large	184	16	92.0	2.82
X-Large	168	32	84.0	4.44
Total	919	81	91.9	38.98 (h)

L’intégration du solveur formel CBMC améliore drastiquement le taux de réussite, passant de 19,2% à 91,9%.

Avantages et limitations de l’approche CBMC **Avantages.** CBMC s’intègre naturellement à l’écosystème C, ce qui évite de changer de langage ou de paradigme. Cette continuité rend l’approche facilement extensible : il est possible de construire un DSL spécifique à un problème donné tout en profitant des outils classiques du langage C (structures, fonctions, macros, etc.). Cela offre une grande flexibilité et permet d’adapter la modélisation à des contextes variés.

Limitations. Cependant, CBMC n’offre pas de langage dédié à la modélisation de contraintes logiques complexes. L’expressivité reste limitée : il est difficile d’exprimer certaines contraintes symboliques de haut niveau de manière concise ou naturelle (par exemple, la contrainte “toutes les personnes ont des tailles différentes”). Contrairement à Prolog, CBMC ne dispose pas d’opérateurs logiques intégrés comme `all_different/1` ou `#==>`, ce qui oblige à écrire ces relations de façon plus verbeuse et moins déclarative.

3.2 Résolution via Prolog et la bibliothèque `clpfd`

Une seconde approche a consisté à modéliser les puzzles logiques en utilisant le langage Prolog, couplé à la bibliothèque `clpfd` (Constraint Logic Programming over Finite Domains)

Modélisation par entiers. Chaque entité du puzzle (personnage, maison, destination, etc.) est représentée par une variable entière comprise entre 1 et n (où n est le nombre de maisons). La valeur d’une variable indique la position de l’entité dans l’ordre des maisons. Par exemple, si `Eric = 2`, cela signifie qu’Eric vit dans la maison 2.

```
solve :-
    % Extract all entities from the puzzle
    Vars = [Eric, Arnold, Beach, Mountain, Short, Average],

    Vars ins 1..2,
    Names = [Eric, Arnold],
    Vacations = [Beach, Mountain],
    Heights = [Short, Average],

    % Add constraints for each group
    all_different(Names),
    all_different(Vacations),
    all_different(Heights),
```

L’utilisation d’entiers permet une compatibilité directe avec les contraintes de `clpfd`, en particulier avec `all_different/1`, qui impose l’unicité des éléments dans une liste.

Encodage des contraintes. Les indices associés aux entités permettent d’exprimer les règles du puzzle sous forme de relations d’égalité, d’inégalité ou de comparaison entre entiers, à l’aide des opérateurs autorisés par le DSL Prolog : `#=`, `#\=`, `#<`, `#>` et `abs/1`. Les contraintes sont ainsi exprimées de manière déclarative :

```
% Clue 1: Arnold is not in the third house.
Arnold #\= 3,

% Clue 2: Arnold is the person who enjoys mountain retreats.
Arnold #= Mountain.
```

Affichage de la solution. La solution retournée par Prolog est une liste d’entiers correspondant à la liste des variables `Vars`. Ces entiers sont ensuite traités pour reconstruire une sortie structurée où chaque maison est décrite par les entités qui lui sont associées.

Performances expérimentales. Cette approche a été testée sur les mêmes puzzles que ceux du benchmark Zebra. Les résultats moyens selon la taille des puzzles sont présentés ci-dessous :

TABLE 3 – Résultats du pipeline LLM + Prolog (`clpfd`)

Catégorie	Succès	Échecs	Taux de réussite (%)	Temps moyen (m)
Small	135	25	84.4	1.08
Medium	123	17	87.9	1.65
Large	90	10	90.0	2.22
X-Large	75	25	75.0	3.15
Total	423	77	84.6	15.65 (h)

Le taux de réussite global atteint 84,6%, avec un taux de succès par cellule de 88,07%. Les performances se dégradent légèrement pour les puzzles les plus volumineux, mais restent significativement supérieures à celles obtenues par le LLM seul.

Avantages et limitations de l’approche Prolog L’approche Prolog offre un cadre expressif et naturel pour la modélisation de contraintes, tout en assurant une exécution rapide. La représentation est très simple et beaucoup plus expressive que celle proposée par CBMC. De plus, le DSL n’est pas limité : il permet même de manipuler la logique du premier ordre, car `clpfd` supporte des opérateurs tels que `#==>` (implication) et `#<=>` (équivalence).

Cependant, plusieurs limitations ont été rencontrées dans la pratique. Premièrement, l’affichage des résultats est actuellement réalisé manuellement. En effet, il est difficile pour un modèle de langage (LLM) de générer automatiquement les fonctions de correspondance entre les variables, qui sont représentées par des entiers, et les constantes symboliques (par exemple, `alice`). Pour ce faire, il faudrait définir des relations telles que `label1(1, alice)` et utiliser des prédicats comme `nth1` pour retrouver le symbole correspondant à un indice numérique, ce qui complique la traduction des solutions en une forme lisible.

Deuxièmement, un autre problème important vient de l’utilisation parfois inappropriée des opérateurs logiques dans `clpfd`. Par exemple, certains exemples utilisent l’opérateur Prolog classique `->` (implication), ce qui génère l’erreur suivante :

```
ERROR: -g solve: Domain error: ‘clpfd_reifiable_expression’ expected,
found ‘(_3794#=1->_4050#=1)#\/_3794#=2->_4050#=2)’
```

Remplacer `->` par `#==>` permet de corriger cette erreur.

Enfin, certains exemples modélisent les contraintes de façon trop compliquée. Par exemple :

```
% Clue 2: Arnold is the person who enjoys mountain retreats.
(Arnold #= 1 -> Mountain #= 1) #\/_
(Arnold #= 2 -> Mountain #= 2) #\/_
(Arnold #= 3 -> Mountain #= 3)
```

Cette contrainte peut être simplifiée en une seule expression :

```
Arnold #= Mountain
```

Le recours systématique à des formulations complexes combinées à l’utilisation d’opérateurs inadaptés complique la modélisation et ralentit la résolution. Une formulation concise et conforme permet un traitement plus efficace.

TABLE 4 – Comparaison synthétique des taux de réussite et temps totaux

Méthode	Taux de réussite (%)	Temps total (h)
LLM seul	19.2	6.74
LLM + CBMC	91.9	38.98
LLM + Prolog	84.6	31.31

4 Comparaison globale

5 Améliorations réalisées

Plusieurs améliorations sont envisagées pour rendre le pipeline plus robuste et performant :

- **Communication entre le solveur et le LLM de correction** : En cas d’erreur détectée par le solveur formel (CBMC), cette erreur sera transmise à un LLM spécialisé qui reformulera le message, expliquera la faute dans le code généré, et proposera des pistes de correction. Cette rétroaction sera renvoyée au LLM principal qui génère le programme DSL, afin d’améliorer la génération.

Pour pallier les échecs, la température du modèle pourra être ajustée dynamiquement lors des appels au LLM, favorisant une diversité accrue ou une meilleure précision dans la génération. Cette approche vise à déterminer la température optimale pour notre pipeline.

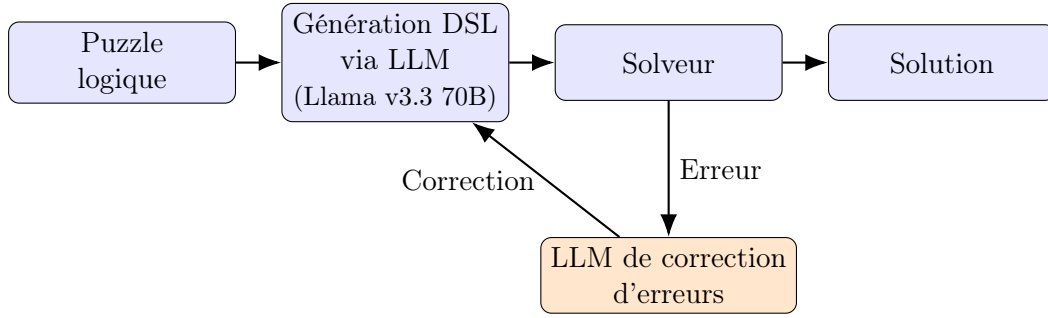


FIGURE 2 – Extension du pipeline avec pré-traitement et correction d’erreurs

- **Amélioration automatique des prompts par itération contrôlée** : Une procédure expérimentale a été mise en place pour corriger automatiquement les prompts lorsque le code généré échoue (erreur d’exécution ou solution incorrecte). Le LLM reçoit en entrée le prompt initial, le code généré, et l’erreur rencontrée. Il propose alors une version modifiée du prompt. Cette tentative est répétée jusqu’à trois fois. Si aucune solution correcte n’est trouvée, la température du LLM est augmentée, et une nouvelle série de trois tentatives est lancée (jusqu’à trois niveaux de température différents). Cela forme une boucle d’amélioration 3×3 (trois itérations pour trois températures différentes). Dès qu’une solution est correcte, le prompt est mis à jour définitivement, la température est remise à zéro, et le pipeline passe au puzzle suivant.

6 Perspectives d’amélioration

6.1 Limites de l’amélioration locale des prompts et approche globale

Cette méthode itérative améliore localement les prompts, mais ne garantit pas leur robustesse globale. Une version corrigée peut résoudre un puzzle donné sans pour autant conserver les bonnes performances sur les autres. Une amélioration locale peut donc introduire des effets de bord sur d’autres cas du dataset.

Pour limiter ces effets, On peut voir :

- *Évaluation croisée* : le prompt corrigé est testé sur les puzzles déjà résolus pour s’assurer qu’il ne provoque pas de régressions.

- *Score global* : chaque prompt est évalué par un score correspondant au nombre total de puzzles correctement résolus. Une version n'est acceptée que si elle améliore ou maintient ce score.

Une alternative consiste à adopter une approche *globale* de correction. Le prompt initial est d'abord exécuté sur l'ensemble du dataset. Les cas d'échec sont ensuite collectés (erreurs d'exécution, mauvaises sorties, etc.) et présentés conjointement au LLM, accompagné du prompt initial, afin qu'il propose une version modifiée permettant de corriger un maximum d'erreurs.

Une version optimisée de cette stratégie consiste à boucler uniquement sur les puzzles en échec. Le LLM ne reçoit alors que les erreurs détectées, ce qui permet une correction progressive mais globale, sans repasser inutilement sur les puzzles déjà validés. Afin de garantir que les corrections proposées ne détériorent pas les performances antérieures, une évaluation croisée partielle est réalisée sur un *batch de validation* extrait aléatoirement parmi les puzzles précédemment résolus. Cela permet de vérifier la robustesse du prompt corrigé tout en limitant le coût de réévaluation. Seules.

6.2 Optimisation de la vitesse avec les coroutines

L'inférence repose actuellement sur la bibliothèque `transformers`, en combinaison avec une classe nommée `AsyncPool`, qui permet de limiter le nombre de coroutines exécutées en concurrence. Cette limite est pour l'instant fixée à 1, ce qui revient à exécuter les inférences de manière séquentielle. Ce choix a été fait après qu'une tentative d'augmentation du nombre de coroutines a provoqué un plantage de la machine, alors même que celle-ci dispose de ressources matérielles suffisantes.

Une première piste d'amélioration consiste donc à retenter l'augmentation progressive du nombre de coroutines, afin de déterminer si le problème initial persiste ou s'il était ponctuel.

En complément, une alternative a été testée en remplaçant `transformers` par `Ollama`, qui supporte naturellement l'exécution asynchrone. Sur mon PC personnel, l'exécution asynchrone fonctionne bien avec `Ollama`, mais j'ai remarqué que c'était un peu plus lent que le mode séquentiel. Il reste à vérifier si cette différence vient de la bibliothèque elle-même ou aux limites de mon PC. Il serait donc pertinent de tester `Ollama` sur la machine pour comparer les résultats.

7 Gestion de l'inférence et optimisation du benchmark

7.1 Optimisation mémoire par quantification

L'inférence est réalisée localement sur un modèle LLaMA v3.3 70B Instruct quantifié en 4-bit grâce à la bibliothèque `bitsandbytes`, permettant une réduction significative de la mémoire vidéo utilisée, tout en conservant une bonne précision. Ce modèle très volumineux (~44Go VRAM en 4-bit) est réparti sur deux GPU NVIDIA RTX™ A6000, chacun disposant de 48 Go de VRAM

7.2 Mécanismes de génération textuelle : température, sampling et beam search

Lors de la génération de texte avec un modèle de langage comme LLaMA, la production de mots repose sur des techniques probabilistes qui dépendent principalement de la distribution de probabilité sur le vocabulaire à chaque étape de génération. Cette distribution est obtenue à l'aide d'une fonction *softmax*, souvent combinée à des techniques de filtrage ou de recherche.

La fonction softmax et la température La fonction *softmax* transforme les logits (valeurs réelles non normalisées issues du modèle) en une distribution de probabilité sur le vocabulaire :

$$\text{softmax}(z_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

où z_i est le logit du mot i , et T la **température**. Lorsque T est bas ($T < 1$), la distribution devient plus concentrée sur les mots les plus probables. À l'inverse, une température élevée ($T > 1$) lisse la distribution, augmentant ainsi la diversité au prix de la cohérence.

Dans le cadre du modèle LLaMA v3.3 70B Instruct, la température maximale est $T = 1$.

Quand $T \rightarrow 0$, le modèle devient déterministe : il choisit toujours le mot ayant la probabilité maximale. On parle alors de **greedy decoding** (décodage glouton).

Pour une présentation visuelle de la température et de ses effets, voir [1].

Méthodes de sampling : top- k et top- p Lorsque la température est non nulle, on peut appliquer des techniques de *sampling* pour introduire de la stochasticité :

- **top- k sampling** : on trie les tokens par probabilité décroissante et on conserve uniquement les k premiers. Ensuite, on recalcul la distribution de probabilité (avec softmax) uniquement sur ce sous-ensemble réduit, et on effectue un échantillonnage selon cette nouvelle distribution.
- **top- p sampling** : on trie également les tokens par probabilité décroissante, puis on conserve les tokens dont la somme cumulative des probabilités atteint un seuil p (par exemple $p = 0.9$). La distribution est ensuite re-normalisée sur ce sous-ensemble, avant d'échantillonner.
Le paramètre p contrôle la taille du “noyau” de vocabulaire à considérer :
- p proche de 1 (ex. 0.95) : un grand nombre de tokens est considéré, ce qui augmente la diversité et l'aléa.
- p proche de 0 (ex. 0.1) : seuls les tokens les plus probables sont retenus, rendant la sortie plus déterministe.

Méthode greedy (température = 0) Si la température est strictement nulle, on effectue un simple **argmax** à chaque étape. Le mot ayant la probabilité maximale est sélectionné. Cela permet une génération rapide et déterministe, mais peut produire du texte répétitif ou monotone.

Méthode beam search La méthode *beam search* garde plusieurs hypothèses en parallèle. À chaque étape, elle conserve les B séquences les plus prometteuses (appelées *beams*), qu'elle étend à chaque nouveau mot en explorant leurs suites les plus probables. C'est une technique heuristique souvent plus cohérente que le sampling, au prix d'un coût de calcul plus élevé.

Exemple avec beam_width = 3 :

1. Étape 1 : on génère les 3 mots les plus probables.
2. Étape 2 : pour chaque mot, on génère les 3 suivants (soit 9 séquences candidates), et on conserve les 3 meilleures.
3. Et ainsi de suite jusqu'à atteindre la longueur maximale ou un token de fin.

Résumé

Méthode	Description	Avantages	Inconvénients
Greedy	Sélection du mot avec la plus haute probabilité à chaque étape	Rapide, déterministe	Peu créatif, parfois répétitif
Sampling (top- k , top- p , temp)	Échantillonnage basé sur la distribution des probabilités	Créatif, diversité	Moins stable, parfois incohérent
Beam search	Recherche parallèle des meilleures séquences	Plus structuré, souvent cohérent	Lourd en calcul, peut manquer de diversité

Intégration dans un pipeline transformers Voici un exemple d'utilisation de ces méthodes dans un pipeline Python :

```

from transformers import pipeline

pipe = pipeline("text-generation", model=model, tokenizer=tokenizer, device=0)

# Avec sampling
output = pipe(
    "Once upon a time",
    do_sample=True,
    temperature=0.8,
    top_k=50,
    top_p=0.9,
    max_new_tokens=100
)

# Avec beam search
output = pipe(
    "Once upon a time",
    do_sample=False,
    num_beams=5,
    max_new_tokens=100
)

```

Pour une explication interactive des méthodes greedy, beam et sampling, voir [2].

Références

- [1] Aviral Kumar, *Understanding LLM Parameters : Temperature, Top-K, Top-P*, <https://aviralrma.medium.com/understanding-llm-parameters-c2db4b07f0ee>
- [2] Niklas Heidloff, *Greedy, Beam Search and Sampling Decoding*, <https://heidloff.net/article/greedy-beam-sampling/>

8 Research Papers Consulted

- **Reliable Reasoning Beyond Natural Language**
ce travail propose un pipeline où un LLM génère du code Prolog à partir d'un énoncé naturel, et un interpréteur Prolog exécute ce code pour produire une réponse exacte. Si l'exécution échoue, le LLM relance plusieurs tentatives jusqu'à obtenir un code exécutable.
- **Solving Zebra Puzzles Using Constraint-Guided Multi-Agent Systems**
ce travail propose un système multi-agent (ZPS) combinant LLMs et solveur SMT pour décomposer et résoudre le Zebra Puzzle, avec une amélioration de 166% des solutions complètes pour GPT-4 grâce à une boucle de rétroaction entre agents.
- **ZebraLogic : On the Scaling Limits of LLMs for Logical Reasoning**
présente un benchmark ZebraLogic testant les capacités de raisonnement logique des LLMs sur des puzzles de complexité croissante, révélant une baisse marquée de précision à mesure que la difficulté augmente, ce qu'ils nomment la «curse of complexity »
- **Combining Constraint Programming Reasoning with Large Language Model Predictions**
propose GenCP, une méthode qui intègre un LLM dans un cadre de Programmation par Contraintes (CP) pour générer du texte tout en respectant des contraintes structurelles robustes